Towards a Theory of Virtual Machines

A Working Draft

June 5, 2026

Contents

1 Introduction

This document is a working sketch of a unified theory of virtual machines. The goal is to build upward from the most primitive mathematical ingredients — syntax, semantics, transition systems — through abstract machines, and eventually toward a formal account of layering, compilation correctness, and the properties that distinguish a virtual machine from a mere interpreter.

The document is intentionally incomplete. Sections marked *(to be developed)* are placeholders for future work.

2 Syntax: Languages as Inductive Sets

Before any notion of computation, we need a *language* — a set of well-formed expressions. At its most abstract, a language is defined by a grammar, typically given as a BNF or as an inductive definition.

Definition 2.1 (Abstract syntax)**.** An *abstract syntax* is an inductive set $\mathcal{E}$ generated by a finite set of constructors, each with a fixed arity. A grammar

$$e \ ::= \ n \ \mid \ e_1 + e_2 \ \mid \ e_1 \times e_2$$

defines $\mathcal{E}$ as the least set closed under those constructors.

Because $\mathcal{E}$ is freely generated, *structural induction* is available: to prove a property $P(e)$ for all $e \in \mathcal{E}$, it suffices to prove P for each base case and to show each constructor preserves P. This is the principal proof technique throughout the theory.

Remark 2.2. Concrete syntax (parsing, lexing) is deliberately set aside. The theory operates on abstract syntax trees.

3 Semantics

A language without a semantics is merely a set of strings. A semantics assigns *meaning* to expressions. There are three major styles; each will play a role in the theory.

3.1 Denotational Semantics

Definition 3.1 (Denotational semantics)**.** A *denotational semantics* is a compositional mapping

$$[\![\cdot]\!] \ : \ \mathcal{E} \ \to \ D$$

from expressions into a mathematical domain D. Compositionality means that $[\![e]\!]$ is defined solely in terms of $[\![e_i]\!]$ for the immediate subexpressions e_i of e.

For languages with recursion, D must support least fixed points. Domain theory (Scott, Strachey) provides the required structure: complete partial orders and continuous functions.

3.2 Operational Semantics

Rather than mapping to a domain, operational semantics describes *computation steps* via inference rules.

Definition 3.2 (Big-step semantics)**.** A *big-step* (or *natural*) semantics defines a relation $e \Downarrow v$, read "expression e evaluates to value v", by inductive rules. For example:

$$\frac{e_1 \Downarrow n_1 \qquad e_2 \Downarrow n_2}{e_1 + e_2 \Downarrow n_1 + n_2}$$

Definition 3.3 (Small-step semantics)**.** A *small-step* (or *structural operational*) semantics defines a one-step reduction relation $e \to e'$. Full evaluation is the reflexive-transitive closure $\to^*$.

Small-step semantics exposes intermediate states, which makes it the natural starting point for deriving abstract machines.

3.3 Axiomatic Semantics

Hoare (1969) proposed giving meaning to programs via logical assertions about program states. A *Hoare triple* $\{P\}\, c\, \{Q\}$ reads: if assertion P holds before executing command c and c terminates, then assertion Q holds afterward. This is *partial correctness*; *total correctness* adds the requirement that c terminates whenever P holds.

Definition 3.4 (Hoare triple, partial correctness)**.** Let $\mathcal{L}$ be a command language and $\mathcal{A}$ a set of assertions over states. The partial-correctness triple $\{P\}\, c\, \{Q\}$ (with $P, Q \in \mathcal{A}$, $c \in \mathcal{L}$) is *valid* if for every state σ:

$$\sigma \models P \;\text{ and }\; \langle c,\, \sigma \rangle \Downarrow \sigma' \implies \sigma' \models Q.$$

The proof system adds axioms for atomic commands (e.g. the assignment axiom $\{P[e/x]\}\; x := e\; \{P\}$) and inference rules for compound statements. The key structural rule is:

$$\frac{\models P \Rightarrow P' \qquad \{P'\}\, c\, \{Q'\} \qquad \models Q' \Rightarrow Q}{\{P\}\, c\, \{Q\}} \tag{Consequence}$$

Axiomatic semantics is primarily relevant for program verification. It connects to the isolation results of Section 11 via *separation logic* (Reynolds, 2002; O'Hearn et al., 2001), and to Section 14 via proof-carrying code.

3.4 Correspondence Between Semantics

The three styles are not independent. A central obligation of any language definition is to show they agree.

Theorem 3.5 (Big-step / small-step equivalence)**.** *For a well-defined language,*

$$e \Downarrow v \iff e \to^* v.$$

Theorem 3.6 (Operational / denotational agreement)**.**

$$e \Downarrow v \iff [\![e]\!] = [\![v]\!].$$

These correspondence theorems are non-trivial and constitute a significant part of foundational programming language theory. They guarantee that the different views are indeed views of the *same* thing.

4 Transition Systems

Both operational semantics styles are instances of a more primitive structure.

Definition 4.1 (Transition system)**.** A *transition system* is a triple $\mathcal{T} = \langle S, s_0, \to \rangle$ where S is a set of *states*, $s_0 \in S$ is the *initial state*, and $\to \subseteq S \times S$ is the *transition relation*.

A *computation* is a maximal sequence $s_0 \to s_1 \to s_2 \to \cdots$. It is *terminating* if the sequence is finite, ending in a state with no outgoing transitions.

Definition 4.2 (Determinism)**.** A transition system is *deterministic* if $\to$ is a partial function: for each s there is at most one s' with $s \to s'$.

Definition 4.3 (Confluence)**.** A transition system is *confluent* (has the Church–Rosser property) if whenever $s \to^* s_1$ and $s \to^* s_2$, there exists s' such that $s_1 \to^* s'$ and $s_2 \to^* s'$.

Determinism implies confluence, but not conversely. Confluence is the weaker and often more useful property: it says that evaluation order does not affect the final result, even if intermediate paths differ.

5 Evaluation Contexts and Reduction Strategies

Raw small-step reduction is typically non-deterministic: many subterms may be reducible at once. To obtain a machine we must fix a *reduction strategy*.

Definition 5.1 (Evaluation context)**.** An *evaluation context* E is an expression with a distinguished *hole* $[\cdot]$, defined by a grammar that specifies which positions are eligible for reduction. The notation $E[e]$ denotes E with the hole filled by e.

Example 5.2 (Left-to-right call-by-value)**.**

$$E \ ::= \ [\cdot] \ \mid \ E + e \ \mid \ v + E$$

forces the left operand to be reduced first, and requires it to be a value before the right operand is touched.

With evaluation contexts, the reduction rule becomes:

$$\frac{e \to e'}{E[e] \to E[e']}$$

This is now deterministic given the context grammar, and it specifies exactly the evaluation order of the language.

6 Abstract Machines

Definition 6.1 (Abstract machine)**.** An *abstract machine* for a language $\mathcal{E}$ is a deterministic transition system $\mathcal{M} = \langle S, s_0, \to \rangle$ together with:

- an *injection* $\iota : \mathcal{E} \to S$ loading an expression as an initial machine state, and
- an *extraction* $\varepsilon : S_{\text{fin}} \rightharpoonup V$ reading a value out of a terminal state, where $S_{\text{fin}} = \{s \in S \mid \nexists\, s'.\ s \to s'\}$ is the set of *terminal* (stuck or halted) states,

such that $\varepsilon(\iota(e)\!\downarrow) = [\![e]\!]$ for all terminating e (where $\iota(e)\!\downarrow$ denotes the unique terminal state reached from $\iota(e)$ by the $\to^*$ relation).

The equation above is the *correctness condition*: running the machine produces the same result as the semantics.

6.1 The SECD Machine

Landin (1964) introduced the first abstract machine for the lambda calculus. A state is a four-tuple $\langle S, E, C, D\rangle$:

- S — **Stack**: accumulates intermediate values.
- E — **Environment**: a finite map from variables to values.
- C — **Control**: a sequence of instructions to execute.
- D — **Dump**: saved $\langle S, E, C\rangle$ triples, representing the call stack.

The transition rules use four core instructions. $\texttt{LD}(x)$ loads a variable; $\texttt{LDC}(v)$ loads a constant; $\texttt{LDF}(f)$ creates a closure; `AP` applies a closure to the top stack value; and `RTN` returns from a function call:

$$\begin{aligned}
\langle S,\ E,\ [\texttt{LD}(x) \mid C],\ D\rangle &\ \to\ \langle E(x) \cdot S,\ E,\ C,\ D\rangle \\
\langle S,\ E,\ [\texttt{LDC}(v) \mid C],\ D\rangle &\ \to\ \langle v \cdot S,\ E,\ C,\ D\rangle \\
\langle S,\ E,\ [\texttt{LDF}(f) \mid C],\ D\rangle &\ \to\ \langle (f, E) \cdot S,\ E,\ C,\ D\rangle \\
\langle v \cdot (f, E') \cdot S,\ E,\ [\texttt{AP} \mid C],\ D\rangle &\ \to\ \langle [],\ E'[x \mapsto v],\ f,\ \langle S, E, C\rangle \cdot D\rangle \\
\langle v \cdot [],\ E',\ [\texttt{RTN}],\ \langle S, E, C\rangle \cdot D\rangle &\ \to\ \langle v \cdot S,\ E,\ C,\ D\rangle
\end{aligned}$$

The final rule restores the saved context from the dump; the `AP` rule saves the current context before entering the function body.

6.2 The CEK Machine

Felleisen and Friedman (1986) reformulated the machine with an explicit, first-class continuation. A state is a triple $\langle C, E, K\rangle$:

- C — **Control**: current expression or value.
- E — **Environment**: variable bindings (variables to values).
- K — **Kontinuation**: an algebraic representation of the remaining computation.

Continuations are drawn from a small grammar: $K \ ::=\ \texttt{Done} \mid \texttt{EvalArg}(t, E, K) \mid \texttt{Apply}(v, K)$.

The transition rules split into two modes. When C is an expression (evaluation mode), the machine decomposes it; when C is a value (return mode), it dispatches on the continuation:

$$\begin{aligned}
\langle x,\ E,\ K\rangle &\ \to\ \langle E(x),\ E,\ K\rangle \\
\langle \lambda x.\, t,\ E,\ K\rangle &\ \to\ \langle \langle x, t, E\rangle,\ E,\ K\rangle \\
\langle t_1\, t_2,\ E,\ K\rangle &\ \to\ \langle t_1,\ E,\ \texttt{EvalArg}(t_2, E, K)\rangle \\
\langle v,\ E,\ \texttt{EvalArg}(t, E', K)\rangle &\ \to\ \langle t,\ E',\ \texttt{Apply}(v, K)\rangle \\
\langle v,\ E,\ \texttt{Apply}(\langle x, t, E'\rangle,\ K)\rangle &\ \to\ \langle t,\ E'[x \mapsto v],\ K\rangle
\end{aligned}$$

The terminal state is $\langle v, E, \texttt{Done}\rangle$. Making continuations first-class simplifies the treatment of control operators such as `call/cc` and facilitates correctness proofs.

6.3 The Krivine Machine

Designed for call-by-name evaluation of the lambda calculus. The state $\langle t, E, \pi\rangle$ is minimal:

- t — current **term**.
- E — **environment**: maps variables to *thunks* $\langle t', E' \rangle$, i.e. unevaluated (term, environment) pairs.
- π — argument **stack**: a list of thunks waiting to be applied to the current term.

There are exactly three transition rules:

$$\langle x,\ E,\ \pi \rangle \ \to\ \langle t',\ E',\ \pi \rangle \qquad \text{if } E(x) = \langle t', E' \rangle \qquad \text{(Var)}$$

$$\langle t_1\, t_2,\ E,\ \pi \rangle \ \to\ \langle t_1,\ E,\ \langle t_2, E \rangle \cdot \pi \rangle \qquad \text{(App)}$$

$$\langle \lambda x.\, t,\ E,\ \langle t_0, E_0 \rangle \cdot \pi \rangle \ \to\ \langle t,\ E[x \mapsto \langle t_0, E_0 \rangle],\ \pi \rangle \qquad \text{(Lam)}$$

The terminal state is $\langle \lambda x.\, t, E, [] \rangle$ (a *head normal form*: a lambda with no pending arguments).

The key difference from the CEK machine is that the App rule pushes an *unevaluated* thunk $\langle t_2, E \rangle$ rather than evaluating t_2 first. This makes the machine call-by-name: arguments are only forced when a variable is looked up (Var rule). An unused argument is pushed onto π and discarded when the Lam rule extends the environment, so it is never forced at all. Its simplicity makes it a clean target for formal proofs of evaluation strategy correctness.

6.4 Deriving Machines from Semantics

Danvy and Nielsen (2003); Danvy (2004) showed that abstract machines are not invented ad hoc but can be *calculated* from a reduction semantics by a sequence of meaning-preserving program transformations:

1. Start with a compositional big-step (or small-step) evaluator.
2. Apply a *continuation-passing style* (CPS) transformation to make control flow explicit as a higher-order function argument.
3. *Defunctionalize* the continuations: replace the higher-order continuation functions with an algebraic data type of constructors, each corresponding to one point where the continuation is called.
4. The result is an abstract machine whose correctness is guaranteed by the correctness of each transformation step.

Example 6.2 (Deriving the CEK machine)**.** Start from a call-by-value evaluator (written in direct style):

$$\begin{aligned} \mathrm{eval}(x,\ E) &= E(x) \\ \mathrm{eval}(\lambda x.\, t,\ E) &= \langle x, t, E \rangle \\ \mathrm{eval}(t_1\, t_2,\ E) &= \mathrm{apply}(\mathrm{eval}(t_1, E),\ \mathrm{eval}(t_2, E)). \end{aligned}$$

After CPS transformation, each call receives an extra argument k (the continuation):

$$\begin{aligned} \mathrm{eval}_k(x,\ E,\ k) &= k(E(x)) \\ \mathrm{eval}_k(\lambda x.\, t,\ E,\ k) &= k(\langle x, t, E \rangle) \\ \mathrm{eval}_k(t_1\, t_2,\ E,\ k) &= \mathrm{eval}_k(t_1,\ E,\ \lambda v.\ \mathrm{eval}_k(t_2,\ E,\ \lambda a.\ \mathrm{apply}_k(v, a, k))) \,. \end{aligned}$$

Three distinct continuation lambdas appear. Defunctionalizing them introduces constructors `Done`, `EvalArg`(t, E, k), and `Apply`(v, k), with an *apply* dispatch function replacing each λ-call. The pair $(\mathrm{eval}_k, \mathrm{apply}_k)$, now operating on these constructors, is precisely the CEK machine of Section 6.2.

This derivation works in both directions: given a machine, one can *reconstruct* the semantics it implements by inverting CPS and defunctionalization. The SECD machine corresponds to a direct-style evaluator with an explicit call stack (Danvy, 2004), and the Krivine machine to a call-by-name evaluator with thunked arguments.

7 Virtualisation of Hardware: The Popek–Goldberg Theorem

Stepping toward system-level virtual machines, the most foundational result is due to Popek and Goldberg (1974).

Definition 7.1 (Instruction classes)**.** Given an instruction set architecture (ISA) with a privilege hierarchy:

- A *privileged instruction* traps when executed in user mode.
- A *sensitive instruction* is one whose behaviour depends on the current privilege level or on privileged machine state.

Theorem 7.2 (Popek–Goldberg)**.** *A virtual machine monitor (VMM) can be constructed for an ISA if and only if the set of sensitive instructions is a subset of the set of privileged instructions.*

Intuitively: if every instruction that *matters* (is sensitive) also *traps* (is privileged), then the VMM can intercept and emulate all guest behaviour. The classic x86 architecture violated this condition, requiring binary translation or paravirtualisation as workarounds.

8 Towards a Unified Framework

The preceding sections establish two largely separate theoretical traditions:

Tradition	Objects	Core tools
Abstract machine theory	SECD, CEK, Krivine, …	Transition systems, operational semantics, CPS, defunctionalization
Virtualization theory	Hypervisors, system VMs	ISA models, privilege levels, Popek–Goldberg

A unified theory of virtual machines would need to account for:

1. **Abstraction layers.** A formal notion of one machine running *on top of* another: what the guest observes versus what the host provides.
2. **Compilation and translation.** Correctness of the mapping from guest instructions to host instructions or actions.
3. **Isolation.** When can one guest computation interfere with another, and what conditions guarantee non-interference?
4. **Resource mapping.** Time, memory, and I/O as modelled quantities, and their relationship between layers.
5. **Fidelity.** When does a guest program behave identically on the VM as it would on the native machine?

Each of these is the subject of existing but scattered work. The goal of this document is to develop them into a coherent whole. The following sections begin that development.

9 Abstraction Layers

The central idea of a virtual machine is that one machine *presents an interface* that another machine *consumes*. To formalise this we need a notion of interface and of the relationship between machines at adjacent levels.

9.1 Interfaces

Definition 9.1 (Interface)**.** An *interface* $\mathcal{I} = \langle \mathcal{O}, \mathcal{R} \rangle$ consists of:

- a set $\mathcal{O}$ of *operations* (instructions, system calls, API entry points), and
- a set $\mathcal{R}$ of *responses* (return values, observable effects).

An interface defines what a consumer may invoke and what it may observe. It says nothing about *how* those operations are implemented.

Remark 9.2. At the hardware level, $\mathcal{O}$ is the instruction set and $\mathcal{R}$ is the set of resulting register and memory states. At the language VM level, $\mathcal{O}$ is the bytecode instruction set and $\mathcal{R}$ is the set of resulting operand-stack and heap states. The same definition covers both.

9.2 Machines as Interface Transformers

Definition 9.3 (Machine as interface transformer)**.** A *machine* $\mathcal{M}$ at layer k is a transition system that:

- *presents* an interface $\mathcal{I}^+$ upward to its consumers (the *guest interface*), and
- *uses* an interface $\mathcal{I}^-$ downward from its provider (the *host interface*).

We write $\mathcal{M} : \mathcal{I}^- \Rightarrow \mathcal{I}^+$.

A machine $\mathcal{M}$ implements each operation $o \in \mathcal{O}^+$ by a finite sequence of operations drawn from $\mathcal{O}^-$, producing a response in $\mathcal{R}^+$ from responses in $\mathcal{R}^-$.

9.3 Layer Stacks

Definition 9.4 (Layer stack)**.** A *layer stack* of depth n is a sequence of machines

$$\mathcal{M}_1 : \mathcal{I}_0 \Rightarrow \mathcal{I}_1, \quad \mathcal{M}_2 : \mathcal{I}_1 \Rightarrow \mathcal{I}_2, \quad \ldots, \quad \mathcal{M}_n : \mathcal{I}_{n-1} \Rightarrow \mathcal{I}_n,$$

where $\mathcal{I}_0$ is the *physical* (or lowest trusted) interface and $\mathcal{I}_n$ is the interface visible to the end user or application.

Composition is the key operation: given $\mathcal{M}_1 : \mathcal{I}_0 \Rightarrow \mathcal{I}_1$ and $\mathcal{M}_2 : \mathcal{I}_1 \Rightarrow \mathcal{I}_2$, their composition $\mathcal{M}_2 \circ \mathcal{M}_1 : \mathcal{I}_0 \Rightarrow \mathcal{I}_2$ is again a machine. This gives layer stacks a *categorical* flavour: machines are morphisms and interfaces are objects.

Example 9.5 (A three-layer stack)**.** A typical execution environment for a managed language:

$$\underbrace{\text{hardware}}_{\mathcal{I}_0} \xrightarrow{\mathcal{M}_1 = \text{OS}} \underbrace{\text{system calls}}_{\mathcal{I}_1} \xrightarrow{\mathcal{M}_2 = \text{JVM}} \underbrace{\text{bytecode}}_{\mathcal{I}_2} \xrightarrow{\mathcal{M}_3 = \text{JIT}} \underbrace{\text{Java}}_{\mathcal{I}_3}$$

9.4 Fidelity

A key property of a layer is that it should behave, from the guest's perspective, *as if* the guest interface were directly available.

Definition 9.6 (Fidelity)**.** A machine $\mathcal{M} : \mathcal{I}^- \Rightarrow \mathcal{I}^+$ has *fidelity* if for every sequence of operations $o_1, o_2, \ldots, o_k \in \mathcal{O}^+$ and every sequence of responses $r_1, \ldots, r_k \in \mathcal{R}^+$, the responses observed through $\mathcal{M}$ are identical to those that would be observed on a native implementation of $\mathcal{I}^+$.

This is a behavioural equivalence condition. The appropriate mathematical tool for establishing it is *bisimulation*, to be developed in a later section.

10 Compilation Correctness

A compiler is the mechanism that maps programs at one layer to programs at the layer below. Its correctness is the formal guarantee that the layer stack preserves meaning.

10.1 Compilers as Semantic-Preserving Maps

Definition 10.1 (Compiler)**.** A *compiler* from source language $\mathcal{E}_S$ to target language $\mathcal{E}_T$ is a total function

$$\mathcal{C} : \mathcal{E}_S \to \mathcal{E}_T.$$

Definition 10.2 (Semantic preservation)**.** Let $[\![\cdot]\!]_S$ and $[\![\cdot]\!]_T$ be the semantics of the source and target languages respectively, over a shared value domain V. The compiler $\mathcal{C}$ is *semantically correct* if for all $e \in \mathcal{E}_S$:

$$[\![\mathcal{C}(e)]\!]_T \;=\; [\![e]\!]_S.$$

This is the fundamental correctness statement: compiling and then running gives the same answer as running the source directly. It says nothing about *how many steps* are taken, only about the final value.

10.2 Simulation Relations

Denotational agreement is clean but coarse. For abstract machines, where we care about intermediate states, we need a finer tool.

Definition 10.3 (Simulation)**.** Let $\mathcal{M}_S = \langle S_S, s_0^S, \to_S \rangle$ and $\mathcal{M}_T = \langle S_T, s_0^T, \to_T \rangle$ be transition systems. A relation $R \subseteq S_S \times S_T$ is a *simulation* of $\mathcal{M}_S$ by $\mathcal{M}_T$ if:

1. $\langle s_0^S,\, s_0^T \rangle \in R$, and
2. whenever $\langle s, t \rangle \in R$ and $s \to_S s'$, there exists t' such that $t \to_T^+ t'$ and $\langle s', t' \rangle \in R$.

Condition (2) is the *simulation step*: every source step is matched by one or more target steps, landing in a related state. The use of $\to_T^+$ (one or more steps) rather than $\to_T$ (exactly one) allows the target to take several small steps to implement a single source step—which is exactly what happens in compilation.

Definition 10.4 (Bisimulation)**.** A relation R is a *bisimulation* if both R and its converse R^{-1} are simulations. Two machines are *bisimilar* if there exists a bisimulation relating them.

Bisimulation is the standard notion of *behavioural equivalence* for transition systems. Two bisimilar machines are indistinguishable by any observer that can only see transitions—they are, in the relevant sense, *the same machine at different levels of description.*

10.3 Compiler Correctness via Simulation

Theorem 10.5 (Compiler correctness, operational form)**.** *Let $\mathcal{C}$ be a compiler from $\mathcal{E}_S$ to $\mathcal{E}_T$, and let $\mathcal{M}_S$, $\mathcal{M}_T$ be the respective abstract machines. Suppose there exists a simulation R of $\mathcal{M}_S$ by $\mathcal{M}_T$ such that $\langle \iota_S(e),\, \iota_T(\mathcal{C}(e))\rangle \in R$ for all e. Then $\mathcal{C}$ is semantically correct.*

The proof proceeds by induction on computation length in $\mathcal{M}_S$: each source step is followed through R to a target step sequence, and terminal states are mapped to equal values by the extraction functions ε_S and ε_T.

10.4 Composing Correct Compilers

A crucial property for layer stacks is that correctness composes.

Lemma 10.6 (Composition of simulations)**.** *If R_1 is a simulation of $\mathcal{M}_1$ by $\mathcal{M}_2$ and R_2 is a simulation of $\mathcal{M}_2$ by $\mathcal{M}_3$, then*

$$R_1 \,;\, R_2 \;=\; \{\langle s, u\rangle \mid \exists\, t.\ \langle s, t\rangle \in R_1 \text{ and } \langle t, u\rangle \in R_2\}$$

is a simulation of $\mathcal{M}_1$ by $\mathcal{M}_3$.

This means: if every layer in a stack has a correct compiler to the layer below, then the entire stack is correct end-to-end. Correctness of the whole follows from correctness of the parts. This compositionality result is the theoretical basis for trusting deep software stacks.

11 Isolation and Non-Interference

A virtual machine typically hosts multiple guests, or multiple computations within a single guest. A central security and correctness requirement is that these computations do not interfere with one another. This section develops the formal vocabulary for stating and proving such properties.

11.1 State Partitioning

The simplest form of isolation is *disjoint state*: two computations operate on entirely separate parts of the machine state, so no interaction is possible by construction.

Definition 11.1 (State partition)**.** Let $\mathcal{M} = \langle S, s_0, \to\rangle$ be a machine whose states have the form $s = \langle \sigma_1, \sigma_2, \sigma_c\rangle$, where σ_1 and σ_2 are the *private* state components of two guests and σ_c is *shared* state (if any). The machine has *disjoint-state isolation* if every transition rule either modifies σ_1 alone, modifies σ_2 alone, or reads but does not write σ_c.

Disjoint-state isolation is strong but often too strong: a shared heap, shared file system, or shared network stack means some state is genuinely common. The weaker and more general property is non-interference.

11.2 Non-Interference

Non-interference is a *semantic* property: it says that the *outputs* observable by one principal are unaffected by the *inputs* controlled by another. It was introduced by Goguen and Meseguer (1982) and has since become the standard formal notion of information-flow security.

We model this via a security lattice. For the purposes of this sketch, a two-level lattice suffices: $L < H$ (Low and High). Low outputs must be visible to unprivileged observers; High inputs are secret.

Definition 11.2 (Security labelling)**.** A *security labelling* for a machine $\mathcal{M}$ is a pair of functions $\ell_{\text{in}} : \mathcal{O} \to \{L, H\}$ and $\ell_{\text{out}} : \mathcal{R} \to \{L, H\}$ assigning a security level to each operation and each observable response.

Definition 11.3 (Low-equivalent states)**.** Two states $s, s' \in S$ are *low-equivalent*, written $s \sim_L s'$, if they agree on all state components labelled L: an observer at level L cannot distinguish them.

Definition 11.4 (Non-interference)**.** A machine $\mathcal{M}$ satisfies *non-interference* if for all states s, s' with $s \sim_L s'$ and all sequences of L-level operations $o_1, \ldots, o_k$:

$$\text{outputs}_L(s,\, o_1 \ldots o_k) \;=\; \text{outputs}_L(s',\, o_1 \ldots o_k),$$

where outputs_L collects the L-labelled responses produced during the execution.

In words: starting from two states that look the same to a low observer, and running the same low-level operations, produces the same low-level outputs regardless of what high-level data the states differ on. High secrets cannot influence what low observers see.

11.3 Non-Interference Across Layers

Non-interference must be stated and preserved at every layer of a stack. A machine $\mathcal{M} : \mathcal{I}^- \Rightarrow \mathcal{I}^+$ should propagate security labels across the interface boundary in a consistent way.

Definition 11.5 (Label-preserving machine)**.** A machine $\mathcal{M} : \mathcal{I}^- \Rightarrow \mathcal{I}^+$ is *label-preserving* if there exist labelling ℓ^+ for $\mathcal{I}^+$ and ℓ^- for $\mathcal{I}^-$ such that every implementation of a guest operation $o \in \mathcal{O}^+$ at level $\ell^+(o)$ uses only host operations $o' \in \mathcal{O}^-$ at levels $\ell^-(o') \le \ell^+(o)$.

A label-preserving machine does not secretly escalate the security level of data as it crosses the layer boundary.

Lemma 11.6 (Non-interference across layers)**.** *If each machine in a layer stack is label-preserving and satisfies non-interference at its own level, then the composed stack satisfies non-interference end-to-end.*

11.4 Separation Logic and Heap Isolation

When the shared state is a heap, a powerful tool for reasoning about disjointness is *separation logic* (Reynolds, 2002; O'Hearn et al., 2001). Its central connective is the *separating conjunction*:

$$P * Q$$

which asserts that P and Q hold on *disjoint* portions of the heap. The *frame rule*

$$\frac{\{P\}\, c\, \{Q\}}{\{P * R\}\, c\, \{Q * R\}}$$

states that if a command c establishes Q from P, it does so without touching the heap region described by R—so R is preserved for free.

In the VM setting, separation logic can be used to prove that the heap allocator used by one guest does not alias with the allocator of another, and that the machine's implementation of memory operations respects guest boundaries. This gives a heap-level account of isolation that complements the information-flow account above.

12 Resource Semantics

All of the preceding sections treat computation as if it were free: steps are taken, states are reached, but no account is kept of *how much* it costs. In a real virtual machine, resources—time, memory, I/O bandwidth—are finite and shared. Their management is both a correctness concern (a guest must not exhaust host resources) and a security concern (resource exhaustion is a denial-of-service attack). This section develops the formal tools for reasoning about resources across layers.

12.1 Cost-Annotated Transition Systems

The simplest extension is to annotate each transition with a cost.

Definition 12.1 (Cost-annotated transition system)**.** A *cost-annotated transition system* is a tuple $\mathcal{T} = \langle S, s_0, \xrightarrow{\cdot}, \mathcal{W} \rangle$ where $\mathcal{W}$ is a *cost monoid* (a commutative monoid of resource amounts, e.g. $(\mathbb{N}, +, 0)$) and the transition relation carries a weight: $s \xrightarrow{w} s'$ with $w \in \mathcal{W}$. The *total cost* of a computation $s_0 \xrightarrow{w_1} s_1 \xrightarrow{w_2} \cdots \xrightarrow{w_n} s_n$ is $w_1 + \cdots + w_n$.

Choosing $\mathcal{W} = \langle \mathbb{N}, +, 0 \rangle$ models time steps; choosing a product $\mathbb{N} \times \mathbb{N}$ models both time and space simultaneously; choosing $\mathbb{N}^k$ covers k independent resource dimensions.

12.2 Cost Models

A *cost model* assigns a weight to each operation at an interface.

Definition 12.2 (Cost model)**.** A *cost model* for an interface $\mathcal{I} = \langle \mathcal{O}, \mathcal{R} \rangle$ is a function $\kappa : \mathcal{O} \to \mathcal{W}$ specifying the resource consumption of each operation.

Different layers may use different cost models for the same operations. The interesting question is how costs at one layer relate to costs at the layer below.

12.3 Cost Preservation Across Layers

Definition 12.3 (Cost-preserving compiler)**.** A compiler $\mathcal{C} : \mathcal{E}_S \to \mathcal{E}_T$ is *cost-preserving* with respect to cost models κ_S and κ_T and a scaling factor $c \in \mathbb{N}$ if for all $e \in \mathcal{E}_S$:

$$\mathrm{cost}_T(\mathcal{C}(e)) \;\leq\; c \cdot \mathrm{cost}_S(e),$$

where $\mathrm{cost}_L(p)$ denotes the total cost of running program p under the machine and cost model at layer L.

This says the compiled program is at most c times more expensive than the source—a *constant-factor overhead* guarantee. JIT compilers aim to push c toward 1 for hot paths; interpreters typically have a larger c.

Remark 12.4. Preservation of asymptotic complexity (big-O) is the weaker and more common guarantee: $\mathrm{cost}_T(\mathcal{C}(e)) = O(\mathrm{cost}_S(e))$. Some verified compilers (e.g. CompCert for C (Leroy, 2009)) prove exact step-count bounds.

12.4 Resource Budgets and Isolation

In a multi-guest setting, resource isolation requires that one guest's consumption cannot affect another's. This is formalised via budgets.

Definition 12.5 (Resource budget)**.** A *resource budget* for a guest g is an upper bound $B_g \in \mathcal{W}$ on the total resources g may consume. The machine enforces the budget if it halts or suspends g once $\mathrm{cost}(g) > B_g$.

Budget enforcement connects resource semantics to isolation: if the machine guarantees that each guest stays within its budget, then resource exhaustion by one guest cannot starve another. This is the formal underpinning of CPU scheduling quotas, memory limits, and I/O throttling in real hypervisors and container runtimes.

12.5 Space: Heap Bounds and Garbage Collection

Memory is a resource with two dimensions: *allocation* (how much is requested) and *retention* (how much is live at any point). A cost model that counts allocations without accounting for collection overestimates true memory use.

Definition 12.6 (Live-memory cost)**.** The *live-memory cost* of a computation at a given step is the size of the set of heap cells reachable from the current machine state.

A garbage collector reduces live-memory cost by reclaiming unreachable cells. The correctness condition for a GC is that it does not reclaim reachable cells (safety) and eventually reclaims all unreachable cells (liveness). Both conditions can be stated as properties of the transition system: safety is invariance of the reachable set under GC steps, and liveness is a progress property on the cost metric.

13 Bisimulation Up To Context

The definition of bisimulation given in Section 10 is correct but often impractical: proving that a relation R is closed under all transitions requires a case analysis that grows with the size of the language. The standard remedy is *bisimulation up to*, a family of proof techniques that allow the relation to be established in a smaller, more tractable form while still yielding full bisimilarity.

13.1 Bisimulation Up To Bisimilarity

Definition 13.1 (Bisimulation up to bisimilarity)**.** A relation $R \subseteq S \times S$ is a *bisimulation up to bisimilarity* $\sim$ if whenever $\langle s, t\rangle \in R$ and $s \to s'$, there exists t' such that $t \to^{+} t'$ and $\langle s', t'\rangle \in \sim \,;\, R \,;\, \sim$.

The key result (Sangiorgi, 1996) is that any bisimulation up to bisimilarity is contained in bisimilarity:

Theorem 13.2 (Soundness of bisimulation up to $\sim$)**.** *If R is a bisimulation up to bisimilarity, then $R \subseteq \sim$.*

This means: to prove two states bisimilar, it is enough to find a relation R containing them that satisfies the weaker up-to condition. Already-known bisimilarities can be used freely on both sides.

13.2 Bisimulation Up To Context

In a language with syntactic structure, transitions often involve placing states inside a common *context*—the same surrounding expression applied to both sides. Requiring the full relation to be closed under all contexts is expensive; up-to-context relaxes this.

Definition 13.3 (Context closure)**.** Given a set of contexts $\mathcal{C}$ (expressions with holes), the *context closure* of a relation R is

$$\overline{R}_{\mathcal{C}} \;=\; \{\langle C[s],\, C[t]\rangle \mid C \in \mathcal{C},\ \langle s, t\rangle \in R\}.$$

Definition 13.4 (Bisimulation up to context)**.** A relation R is a *bisimulation up to context* if whenever $\langle s, t\rangle \in R$ and $s \to s'$, there exists t' such that $t \to^{+} t'$ and $\langle s', t'\rangle \in \overline{\sim}_{\mathcal{C}} \,;\, R \,;\, \overline{\sim}_{\mathcal{C}}$.

Soundness requires that the language's transition relation be *compatible* with the context structure—informally, that contexts do not break the argument. This is the central technical condition in the theory of bisimulation up to, and verifying it amounts to checking that the transition system is a congruence with respect to the language's syntactic operators.

13.3 Relevance to VM Correctness

In the VM setting, bisimulation up to context arises naturally when proving compiler correctness for languages with a rich expression structure. A compiled function body may be bisimilar to its source, but the proof is done in isolation; the up-to-context principle then lifts the local result to a global equivalence, covering all calling contexts. This modularity is what makes per-function or per-module verification tractable in practice.

14 Type Theory and Typed Bytecode

Type systems and virtual machines meet at the interface boundary: a bytecode language equipped with a type system gives the machine a static guarantee that certain classes of error cannot arise at run time. This section develops the connection between the type-theoretic and operational perspectives.

14.1 Types as Invariants on States

A type system assigns to each expression a *type* drawn from a set $\mathcal{T}$. In the transition-system view, a type is an *invariant* on the state space: well-typed states satisfy a predicate that is preserved by every transition.

Definition 14.1 (Type assignment)**.** A *type assignment* for a machine $\mathcal{M} =< S, s_0, \to >$ is a partial function $\tau : S \rightharpoonup \mathcal{T}$ such that if $\tau(s) = T$ and $s \to s'$ then $\tau(s')$ is defined.

The condition says: well-typed states evolve to well-typed states. This is the operational reading of *type soundness*.

14.2 The Subject Reduction Theorem

The canonical formulation of type soundness in an operational semantics is the *subject reduction* (or *type preservation*) theorem:

Theorem 14.2 (Subject reduction)**.** *If* $\vdash e : T$ *and* $e \to e'$*, then* $\vdash e' : T$*.*

Combined with a *progress* theorem—well-typed non-values can always take a step—this gives the standard *type safety* result: a well-typed program never reaches a stuck state that is not a value. Stuck states correspond, in the machine model, to *undefined behaviour*: instructions applied to operands of the wrong kind, out-of-bounds memory accesses, and similar faults that a VM must detect and halt.

14.3 Typed Bytecode and Bytecode Verification

A *typed bytecode language* is a target language $\mathcal{E}_T$ equipped with a type system whose type-checking algorithm runs at load time rather than compile time. The JVM's bytecode verifier is the canonical example: it checks, before execution begins, that the operand stack has consistent types at every program point.

Formally, bytecode verification is a *dataflow analysis* over the control-flow graph of the method:

Definition 14.3 (Bytecode verifier)**.** A *bytecode verifier* for language $\mathcal{E}_T$ is an algorithm that, given a program $p \in \mathcal{E}_T$, assigns to each program point ℓ a *type environment* Γ_ℓ describing the types of all live values, such that:

1. *Entry condition*: Γ_{entry} matches the declared parameter types of the method.
2. *Instruction consistency*: at every program point ℓ, each instruction is applied to operands whose types are consistent with Γ_ℓ.
3. *Stability (dataflow fixed point)*: for every control-flow edge $\ell \to \ell'$, the type environment produced by executing the instruction at ℓ in Γ_ℓ is a subtype (in the join semi-lattice) of $\Gamma_{\ell'}$; at join points the assignment must satisfy $\Gamma_{\ell'} \supseteq \bigsqcup_{\ell \to \ell'} \Gamma_\ell$.

Theorem 14.4 (Verifier soundness)**.** *If the bytecode verifier accepts p, then no execution of p reaches a stuck state.*

This is the VM-level type-safety theorem. It transforms the dynamic check—"is this instruction applied correctly?"—into a static one, performed once at load time, making execution faster and security guarantees stronger.

14.4 Proof-Carrying Code

Necula (1997) generalised typed bytecode into *proof-carrying code* (PCC): the compiler emits not just a program but a *formal proof* that the program satisfies a safety policy. The VM's role reduces to *proof checking*, which is fast and simple even when the safety property is far richer than a type system can express.

In the layer-stack framework, PCC sits at the interface boundary $\mathcal{I}^+$: operations carry attached certificates, and the machine $\mathcal{M}$ checks each certificate before executing the operation. Fidelity is then conditional—the machine executes faithfully *given* that the certificate is valid—and correctness of compilation includes the obligation to produce valid certificates for all compiled programs.

Definition 14.5 (Certificate-carrying interface)**.** An interface $\mathcal{I} = \langle \mathcal{O}, \mathcal{R} \rangle$ is *certificate-carrying* with respect to a safety policy Φ if each operation $o \in \mathcal{O}$ is paired with a proof $\pi \vdash \Phi(o)$, and the machine only executes o if π type-checks.

This closes a circle: type theory, which began as an abstract foundation for mathematics, becomes a run-time enforcement mechanism inside a VM.

15 JIT Compilation

A just-in-time (JIT) compiler differs from an ahead-of-time (AOT) compiler in one essential respect: it runs *concurrently* with the program it is compiling. The source program begins executing in an interpreted mode; the JIT observes execution, compiles hot regions to native

code, and replaces the interpreted version with the compiled one — all while the program continues to run. This interleaving makes correctness subtler than for AOT compilation.

15.1 The On-Stack Replacement Problem

The central technical difficulty is *on-stack replacement* (OSR): at the moment the JIT finishes compiling a function, that function may already be running on the interpreter's call stack. The machine must transfer control from the interpreted state to the compiled state mid-execution, mapping interpreter stack frames to compiled activation records.

Formally, OSR requires a *state mapping* between the two machines at an arbitrary intermediate point, not only at function entry.

Definition 15.1 (OSR mapping)**.** Let $\mathcal{M}_I$ be the interpreter and $\mathcal{M}_J$ the JIT-compiled machine for the same source language, each with an observable output function $\mathrm{obs} : S \to O^*$ (finite traces of outputs). An *OSR mapping* is a relation $R_{\mathrm{osr}} \subseteq S_I \times S_J$ such that whenever $\langle s_I, s_J \rangle \in R_{\mathrm{osr}}$, for every maximal computation sequence starting from s_I the observable output trace is equal to the output trace of the corresponding computation from s_J:

$$\forall\, \pi_I \in \mathrm{Runs}(s_I).\ \exists\, \pi_J \in \mathrm{Runs}(s_J).\ \mathrm{obs}(\pi_I) = \mathrm{obs}(\pi_J).$$

This is essentially a bisimulation condition, but restricted to *observable outputs* rather than all transitions, because the intermediate steps of interpretation and compiled execution will generally differ.

15.2 A Formal Model of JIT Execution

We model JIT execution as a *mixed transition system* whose states carry both an interpretable representation and, optionally, a compiled representation of each active function.

Definition 15.2 (JIT transition system)**.** A *JIT transition system* is a tuple $\mathcal{M}_{\mathrm{JIT}} = \langle S_{\mathrm{mix}},\, s_0,\, \to_I,\, \to_J,\, \to_C \rangle$ where:

- $\to_I$ are *interpreter steps*: execution of one bytecode instruction in the interpreted mode.
- $\to_J$ are *compiled steps*: execution of one native instruction in a JIT-compiled frame.
- $\to_C$ are *compilation steps*: the JIT compiles a region and installs the result, updating the mixed state.

A mixed state $s \in S_{\mathrm{mix}}$ records, for each active function, whether it is currently running in interpreted or compiled mode.

Definition 15.3 (JIT correctness)**.** A JIT system is *correct* if its mixed transition system is observationally equivalent to the pure interpreter: for every program p, the sequence of observable outputs produced by $\mathcal{M}_{\mathrm{JIT}}$ is identical to that produced by $\mathcal{M}_I$.

The proof obligation decomposes into three parts:

1. **AOT correctness of the JIT compiler.** For each compiled region, the compiled code is a correct translation of the source bytecode — the standard simulation relation of Section 10.
2. **OSR correctness.** The OSR mapping R_{osr} is valid at every point where a transition $\to_C$ may occur.

3. **Speculation correctness.** Many JIT compilers emit *speculative* optimisations based on observed run-time behaviour, with *deoptimisation* fallback paths. Each speculation must be guarded by an explicit validity check, and the deoptimisation path must restore a valid interpreter state.

15.3 Speculative Optimisation and Deoptimisation

Speculative compilation is the mechanism that makes JIT compilation fast. The compiler assumes a property P holds (e.g. a particular call site always receives an integer) and emits fast code conditional on P. If P is later violated, a *deoptimisation* event transfers control back to the interpreter.

Definition 15.4 (Speculation triple)**.** A *speculation triple* $\langle G, c_{\text{fast}}, c_{\text{slow}} \rangle$ consists of a *guard* G (a runtime test), a *fast path* c_{fast} valid when G holds, and a *slow path* c_{slow} (the deoptimised fallback). Correctness requires:

$$[\![c_{\text{fast}}]\!] \;=\; [\![c_{\text{slow}}]\!] \;=\; [\![c_{\text{source}}]\!],$$

i.e. both paths are semantically equivalent to the source regardless of whether G holds.

Under this condition, speculative compilation is a refinement of the general compilation correctness theorem: the compiled program is semantically correct under all inputs, not merely under the assumed guard.

16 Hypervisors as Layer-Stack Instances

We now revisit the Popek–Goldberg theorem of Section 7 from inside the unified framework developed in Sections 9 through 15. A hypervisor is a machine in the sense of Definition 8.2, and its correctness conditions are instances of the general definitions already given.

16.1 The Hypervisor as a Machine

A *type-1 hypervisor* (bare-metal) sits directly on hardware and presents to each guest an interface that looks like a physical machine:

$$\mathcal{H} \;:\; \mathcal{I}_{\text{hw}} \;\Rightarrow\; \mathcal{I}_{\text{vm}} \times \cdots \times \mathcal{I}_{\text{vm}},$$

where $\mathcal{I}_{\text{hw}}$ is the physical hardware interface and each $\mathcal{I}_{\text{vm}}$ is a virtual machine interface presented to one guest. The hypervisor multiplexes a single physical interface into n virtual copies.

Definition 16.1 (Hypervisor fidelity)**.** A hypervisor $\mathcal{H}$ has *fidelity* for guest i if the interface $\mathcal{I}_{\text{vm}}^{(i)}$ presented to guest i is bisimilar to a native execution of $\mathcal{I}_{\text{hw}}$ restricted to the resource partition assigned to guest i.

This is exactly the fidelity condition of Definition 8.7, instantiated at the hardware level. Popek–Goldberg gives a sufficient condition on the ISA for this to be achievable via trap-and-emulate alone.

16.2 Trap-and-Emulate as Simulation

The trap-and-emulate mechanism can be read as a simulation in the sense of Section 10. Let the simulation relation R be:

$$< s_{\text{guest}},\, s_{\text{host}} > \in R \;\text{ iff }\; s_{\text{host}} \text{ is a valid VMM state representing } s_{\text{guest}}.$$

The Popek–Goldberg condition guarantees that for every sensitive instruction executed by the guest—which would, if executed natively, alter privileged state—a trap fires, handing control to the VMM. The VMM then emulates the instruction's effect in the host state, restoring the simulation relation. Non-sensitive instructions execute natively without breaking R.

Theorem 16.2 (Trap-and-emulate correctness)**.** *If an ISA satisfies the Popek–Goldberg condition, then the trap-and-emulate hypervisor construction yields a simulation R of the guest machine by the host machine, and therefore the hypervisor has fidelity.*

16.3 Resource Isolation in the Hypervisor

Hypervisor resource isolation is the budget-enforcement requirement of Section 12 instantiated at the hardware level.

- **CPU time** is managed by the VMM scheduler, which assigns each guest a time slice B_g^{cpu} per scheduling period. The scheduler enforces the budget by preempting the guest when the slice expires, using a hardware timer interrupt — itself a privileged, sensitive instruction that always traps.
- **Memory** is managed by the *second-level address translation* (SLAT, also called EPT or NPT) hardware mechanism, which maps guest physical addresses to host physical addresses. Each guest is assigned a fixed page table, bounding its memory budget B_g^{mem} structurally: it cannot address memory outside its assigned pages.
- **I/O** is managed by virtualised device models and, in modern hardware, by an IOMMU that restricts which DMA addresses a guest's devices may access.

Each of these corresponds to a budget $B_g \in \mathcal{W}$ for the appropriate cost dimension, enforced by a combination of hardware traps and VMM policy.

16.4 Para-Virtualisation and Interface Redesign

When the Popek–Goldberg condition is not met—as with the original x86 ISA, which had sensitive instructions that did not trap—two remedies exist:

1. **Binary translation.** The VMM scans guest code before execution and rewrites non-trapping sensitive instructions into explicit traps. This is a dynamic compilation step in the sense of Section 15: the VMM acts as a JIT compiler with the special property that the "optimisation" is actually a *correctness fix* rather than a performance improvement.
2. **Para-virtualisation.** The guest OS is modified to replace sensitive operations with explicit *hypercalls*—operations in $\mathcal{O}^+$ specifically designed to be implementable by the VMM. This is an interface redesign: $\mathcal{I}_{\text{vm}}$ is no longer identical to $\mathcal{I}_{\text{hw}}$ but is a new interface designed for virtualisability.

Para-virtualisation sacrifices fidelity (the guest no longer runs an unmodified OS) in exchange for implementability and performance. In the layer-stack framework, it corresponds to inserting an explicit *adapter layer* between $\mathcal{I}_{\text{hw}}$ and $\mathcal{I}_{\text{vm}}$, and proving correctness of the adapter rather than fidelity of the entire interface.

17 Concurrent and Probabilistic Extensions

All of the preceding sections assume a deterministic, sequential transition system. Real virtual machines are neither: they schedule multiple threads, handle asynchronous events, and in some settings (randomised algorithms, probabilistic bytecode) must reason about probability distributions over outcomes. This section sketches the principal extensions needed.

17.1 Concurrent Transition Systems

Definition 17.1 (Labelled transition system)**.** A *labelled transition system* (LTS) is a triple $\langle S, \Lambda, \rightarrow \rangle$ where Λ is a set of *action labels* and $\rightarrow \subseteq S \times \Lambda \times S$. We write $s \xrightarrow{\alpha} s'$ for $\langle s, \alpha, s' \rangle \in \rightarrow$.

Labels allow transitions to be identified and compared across machines. The label τ (the *silent action*) denotes an internal step not visible to an external observer.

Definition 17.2 (Weak bisimulation)**.** A relation $R \subseteq S \times S$ is a *weak bisimulation* if whenever $\langle s, t \rangle \in R$:

- for every $s \xrightarrow{\alpha} s'$, there exists t' such that $t \xrightarrow{\hat{\alpha}} t'$ and $\langle s', t' \rangle \in R$, where $\xrightarrow{\hat{\alpha}}$ denotes zero or more τ steps followed by α (or just τ^* if $\alpha = \tau$);
- the symmetric condition holds.

Weak bisimulation abstracts over internal computation, identifying machines that agree on all observable actions regardless of how many silent steps intervene. This is essential for compiler correctness in the concurrent setting: a compiled program may take more or fewer internal steps than the source but should produce the same observable behaviour.

17.2 Interleaving Semantics and its Limits

The standard model for concurrency in an LTS is *interleaving*: a concurrent program is modelled as the set of all possible interleavings of its constituent threads. This gives a non-deterministic LTS.

Definition 17.3 (Concurrent composition)**.** The *parallel composition* $P \parallel Q$ of two LTS processes P and Q is the LTS whose states are pairs (s_P, s_Q) and whose transitions are:

$$\frac{s_P \xrightarrow{\alpha} s'_P}{(s_P, s_Q) \xrightarrow{\alpha} (s'_P, s_Q)} \qquad \frac{s_Q \xrightarrow{\alpha} s'_Q}{(s_P, s_Q) \xrightarrow{\alpha} (s_P, s'_Q)} \qquad \frac{s_P \xrightarrow{a} s'_P \quad s_Q \xrightarrow{\bar{a}} s'_Q}{(s_P, s_Q) \xrightarrow{\tau} (s'_P, s'_Q)}$$

The third rule models synchronisation: a and $\bar{a}$ are complementary actions that cancel to a silent step.

Interleaving semantics is standard in process algebras (CCS, CSP, π-calculus) and underlies the formal models of concurrent VMs. Its principal limitation is that it does not distinguish between *true parallelism* and *time-sliced concurrency*—a distinction that matters for resource semantics (wall-clock time) but not for functional correctness.

17.3 Non-Interference Under Concurrency

Extending non-interference to concurrent systems is non-trivial. The naive definition—low outputs are the same in all low-equivalent initial states—is broken by *internal timing*: a high-security thread can influence low-security outputs not through data flow but through scheduling behaviour, cache state, or resource contention.

Definition 17.4 (Strong non-interference (concurrent))**.** A concurrent machine satisfies *strong non-interference* if for all pairs of low-equivalent initial states $s \sim_L s'$ and all schedulers Π, the sequences of low-labelled transitions produced are identical.

This is strictly stronger than the sequential definition. It is generally undecidable for arbitrary schedulers but can be established for restricted scheduling disciplines (e.g. round-robin with fixed quanta) using an appropriate bisimulation.

17.4 Probabilistic Transition Systems

For randomised programs and probabilistic VMs, the transition relation must carry probability weights.

Definition 17.5 (Probabilistic transition system)**.** A *probabilistic transition system* (PTS) is a triple $\langle S,\, s_0,\, P \rangle$ where $P : S \times S \to [0, 1]$ is a transition probability function satisfying $\sum_{s'} P(s, s') \leq 1$ for all s (the deficit models non-termination).

Definition 17.6 (Probabilistic bisimulation)**.** A relation $R \subseteq S \times S$ is a *probabilistic bisimulation* if whenever $\langle s, t \rangle \in R$, for every R-equivalence class C:

$$\sum_{s' \in C} P(s, s') \;=\; \sum_{t' \in C} P(t, t').$$

Probabilistic bisimulation (Larsen and Skou, 1991) equates states that assign the same probability mass to each equivalence class of related states. It collapses to ordinary bisimulation when all probabilities are zero or one.

In the VM setting, probabilistic semantics arises in: garbage collectors with randomised scheduling, load balancers, speculative execution with probabilistic branch predictors, and any VM that exposes a source of randomness to the guest. Compiler correctness in this setting requires that the probability distribution over observable outputs is preserved, not merely the set of possible outputs.

17.5 Game Semantics

An alternative to bisimulation for concurrent and higher-order settings is *game semantics* (Abramsky et al., 2000; Hyland and Ong, 2000). A program's meaning is modelled as a *strategy* in a two-player game between the program (Proponent) and its environment (Opponent). Composition of strategies models sequential and parallel composition of programs.

Game semantics is fully abstract for several languages where denotational models had previously failed, including PCF (higher-order functional language) and languages with state and control. In the VM theory context, game semantics offers a natural model for the interaction between a guest and its host: the host is the Opponent, the guest is the Proponent, and the interface $\mathcal{I}$ specifies the moves available to each player. This perspective is promising but not yet fully developed for the VM setting.

18 Open Problems

The following problems represent genuine open research directions rather than settled theory.

1. **A complete formal model of JIT correctness with speculation.** While the ingredients—simulation relations, OSR mappings, speculation triples—are individually understood, a single unified formal framework covering all three simultaneously, with

machine-checked proofs for a realistic JIT, does not yet exist. The closest work is the Graal/GraalVM partial verification efforts and research on Sourir/Sourir-IR, but a complete formal account remains open.

2. **Compositional resource semantics for concurrent VMs.** Resource budgets compose cleanly in the sequential setting (Section 12), but in the concurrent setting, resources are shared and schedules interact. A compositional theory of resource isolation for concurrent guests—one that supports modular reasoning about individual guests without requiring reasoning about all possible interleavings—is an open problem.

3. **Non-interference for realistic hardware.** Real hardware violates the clean separation assumed by the information-flow models above: caches, branch predictors, and out-of-order execution units are shared microarchitectural state that leaks information across security boundaries (Spectre, Meltdown). A formal model that accounts for microarchitectural state and derives conditions on VM design sufficient to prevent such leaks is an active research area without a settled solution.

4. **Categorical foundations.** The layer-stack framework has an evident categorical structure: interfaces are objects, machines are morphisms, and composition is sequential composition of morphisms. Making this precise—defining the appropriate category, identifying the relevant universal properties, and relating it to existing categorical models of computation (traced monoidal categories, comonads for environments, the category of games)—would unify the framework with a larger body of mathematical theory.

5. **Probabilistic non-interference across layers.** Extending the non-interference results of Section 11 to the probabilistic setting requires a quantitative notion of information leakage (channel capacity, min-entropy leakage) rather than a boolean property. How such quantities compose across layers, and what conditions on each layer bound the end-to-end leakage, is an open problem in quantitative information flow theory.

References

Samson Abramsky, Radha Jagadeesan, and Pasquale Malacaria. Full abstraction for PCF. *Information and Computation*, 163(2):409–470, 2000. doi: 10.1006/inco.2000.2930.

Olivier Danvy. A rational deconstruction of Landin's SECD machine. In *Implementation and Application of Functional Languages (IFL)*, volume 3474 of *Lecture Notes in Computer Science*, pages 52–71. Springer, 2004. doi: 10.1007/11431664_4.

Olivier Danvy and Lasse R. Nielsen. A first-order one-pass CPS transformation. In *Theoretical Computer Science*, volume 308, pages 239–257, 2003. doi: 10.1016/S0304-3975(02)00733-8.

Matthias Felleisen and Daniel P. Friedman. Control operators, the SECD-machine, and the λ-calculus. In E. J. Neuhold and G. Chroust, editors, *Formal Description of Programming Concepts III*, pages 193–217. Elsevier, 1986.

C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259.

J. Martin E. Hyland and C.-H. Luke Ong. On full abstraction for PCF: I, II and III. *Information and Computation*, 163(2):285–408, 2000. doi: 10.1006/inco.2000.2917.

Peter J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4): 308–320, 1964. doi: 10.1093/comjnl/6.4.308.

Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52 (7):107–115, 2009. doi: 10.1145/1538788.1538814.

George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 106–119. ACM, 1997. doi: 10.1145/263699.263712.

Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL)*, volume 2142 of *Lecture Notes in Computer Science*, pages 1–19. Springer, 2001. doi: 10.1007/3-540-44802-0_1.

Gerald J. Popek and Robert P. Goldberg. Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, 17(7):412–421, 1974. doi: 10.1145/361011.361073.

John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 55–74. IEEE, 2002. doi: 10.1109/LICS.2002.1029817.

Davide Sangiorgi. A theory of bisimulation for the π-calculus. *Acta Informatica*, 33(1):69–97, 1996. doi: 10.1007/s002360050043.